

Санкт-Петербургский политехнический университет Петра Великого
Институт прикладной математики и механики
Высшая школа прикладной математики и вычислительной физики

Параллельные вычисления

Лабораторная работа №2

«Решение задачи Дирихле для уравнения Пуассона»

Работу выполнила:
студентка II курса
магистратуры
Добрецова Е.В.
Группа:
5040102/40101
Преподаватель:
Козлов К.Н.

Санкт-Петербург
2026

Содержание

1. Постановка задачи	3
2. Алгоритм метода и условия его применимости	3
2.1. Метод Зейделя для задачи Дирихле для уравнения Пуассона	3
2.2. Одномерная столбцовая декомпозиция	4
2.3. Технология MPI	5
3. Предварительный анализ задачи и проверка условий применимости метода	5
4. Тестовый пример с детальными расчётами	5
5. Подготовка контрольных тестов для иллюстрации метода	8
6. Модульная структура программы	8
7. Численный анализ решения задачи	10
7.1. График решения	10
7.2. Исследование параллельной реализации	10
8. Выводы	13
9. Примечания	13

1. Постановка задачи

Требуется решить задачу Дирихле для уравнения Пуассона:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = (2 - y^2)\cos x \quad (1)$$

в области G с границей Γ , заданной следующим образом:

$$\overline{G} = \{(x, y) \mid 0 \leq x \leq \pi, 0 \leq y \leq 1\}; \quad (2)$$

На границе прямоугольника G известно значение решения (т.е. задано граничное условие первого рода):

$$\begin{cases} u(0, y) = y^2, 0 \leq y \leq 1; u(x, 0) = 0, 0 \leq x \leq \pi \\ u(\pi, y) = -y^2, 0 \leq y \leq 1; u(x, 1) = \cos x, 0 \leq x \leq \pi \end{cases} \quad (3)$$

При решении задачи требуется использовать метод Зейделя с одномерной столбцовой декомпозицией для параллелизации вычислений.

Также необходимо провести анализ данной параллельной реализации, выполнив исследования следующих зависимостей:

1. зависимость времени выполнения от размера изображения;
2. зависимость времени выполнения от числа параллельных процессов/потоков;
3. зависимость ускорения от числа параллельных процессов/потоков;
4. зависимость эффективности параллелизации от числа параллельных процессов/потоков.

2. Алгоритм метода и условия его применимости

2.1. Метод Зейделя для задачи Дирихле для уравнения Пуассона

Для численного решения задачи (1) используется равномерная разностная сетка с шагами h_x и h_y по координатным направлениям. В узлах сетки (x_i, y_i) вводится сеточная функция

$$v_{i,j} \approx u(x_i, y_j), \quad i = \overline{1, N_x}, j = \overline{1, N_y},$$

при этом значения решения в граничных узлах считаются известными.

Аппроксимация оператора Лапласа центральными разностями первого порядка приводят к пятиточечному разностному шаблону:

$$\frac{v_{i+1,j} - 2v_{i,j} + v_{i-1,j}}{h_x^2} + \frac{v_{i,j+1} - 2v_{i,j} + v_{i,j-1}}{h_y^2} = f_{i,j} \quad (4)$$

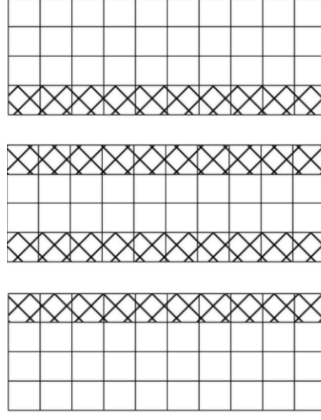


Рис. 2.1. Одномерная столбцовая декомпозиция

Выразим из (4) $v_{i,j}$:

$$v_{i,j} = \frac{\frac{v_{i+1,j} + v_{i-1,j}}{h_x^2} + \frac{v_{i,j-1} + v_{i,j+1}}{h_y^2} - f_{i,j}}{\frac{2}{h_x^2} + \frac{2}{h_y^2}}. \quad (5)$$

Итерационная формула Зейделя для разностной аппроксимации уравнения Пуассона имеет вид:

$$v_{i,j}^{(k+1)} = \frac{\frac{v_{i+1,j}^{(k)} + v_{i-1,j}^{(k+1)}}{h_x^2} + \frac{v_{i,j-1}^{(k)} + v_{i,j+1}^{(k+1)}}{h_y^2} - f_{i,j}}{\frac{2}{h_x^2} + \frac{2}{h_y^2}}, \quad (6)$$

где k - номер итерации, а h_x и h_y - шаги сетки по координатным направлениям.

Итерационный процесс продолжается до выполнения условия сходимости:

$$\max_{i,j} \left| v_{i,j}^{(k+1)} - v_{i,j}^{(k)} \right| < \varepsilon,$$

где ε - заданная точность.

2.2. Одномерная столбцовая декомпозиция

На Рис. 2.1 показана иллюстрация одномерной столбцовой декомпозиции расчётной области, используемая для параллельной реализации метода Зейделя. Исходная область разбивается на вертикальные подобласти, каждая из которых обрабатывается отдельным параллельным процессом. Заштрихованные области соответствуют фиктивным ячейкам, предназначенным для хранения граничных значений сеточной функции, получаемых от соседних процессов.

Декомпозиция области позволяет выполнять пересчёт значений сеточной функции во внутренних узлах каждой подобласти параллельно. Для обеспечения корректности вычислений на каждой итерации осуществляется обмен граничными значениями между соседними подобластями с использованием средств MPI.

2.3. Технология MPI

Для параллельной реализации алгоритма в данной работе используется стандарт MPI (Message Passing Interface), предназначенный для программирования распределённых вычислений. В рамках работы MPI используется для распределения подобластей между процессами, обмена граничными значениями сеточной функции и синхронизации итераций метода Зейделя.

3. Предварительный анализ задачи и проверка условий применимости метода

Исследуемое дифференциальное уравнение в частных производных (1) относится к линейным дифференциальным уравнениям эллиптического типа (уравнение Пуассона). Задача рассматривается в замкнутой прямоугольной области (2).

Функция правой части $f(x, y) = (2 - y^2) \cos(x)$ является непрерывной и бесконечно дифференцируемой в области $\overline{G}$ ($f \in C^\infty$).

Для обеспечения существования классического решения проведена проверка согласованности заданных непрерывных граничных условий Дирихле (3) в угловых точках прямоугольной области $\overline{G}$:

1. Точка $(0, 0)$: $u(0, y)|_{y=0} = 0^2 = 0$ и $u(x, 0) = 0$.
2. Точка $(\pi, 0)$: $u(\pi, y)|_{y=0} = -0^2 = 0$ и $u(x, 0) = 0$.
3. Точка $(0, 1)$: $u(0, y)|_{y=1} = 1^2 = 1$ и $u(x, 1)|_{x=0} = \cos(0) = 1$.
4. Точка $(\pi, 1)$: $u(\pi, y)|_{y=1} = -1^2 = -1$ и $u(x, 1)|_{x=\pi} = \cos(\pi) = -1$.

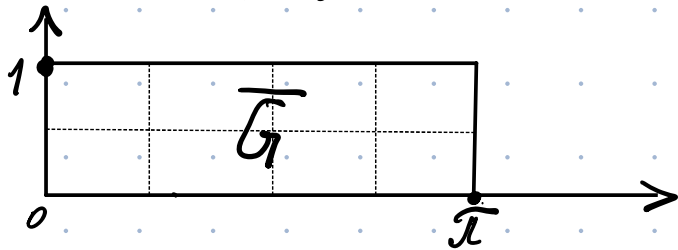
Таким образом, граничные условия непрерывно переходят друг в друга на стыках границ. Следовательно, поставленная краевая задача имеет единственное регулярное решение.

Метод Зейделя применим для решения систем линейных алгебраических уравнений, к которым путём дискретизации сводятся эллиптические дифференциальные уравнения второго порядка. Для задачи Пуассона, аппроксимированной на равномерной сетке, матрица системы обладает диагональным преобладанием и является симметричной и положительно определённой.

4. Тестовый пример с детальными расчётами

Решаем задачу (1) на области (2) с граничным условием (3).

$$\exists N_x = 4, N_y = 2$$



Шаги сетки:

$$h_x = \frac{\pi}{4}, h_y = \frac{1}{2}$$

Узлы сетки:

$$x_i = \frac{\pi i}{4}, i = \overline{0, 4}$$

12 внеш. узлов, 3 внутр.

$$y_i = \frac{i}{2}, i = \overline{0, 2}$$

Граничные значения находим из усл. (3):

$$1) v_{0,0} = u(0,0) = 0$$

$$2) v_{0,1} = u(0,0,5) = 0,25$$

$$3) v_{0,2} = 1^2 = \cos 0 = 1$$

$$4) v_{1,2} = \cos \frac{\pi}{4} = \frac{\sqrt{2}}{2} \approx 0,707$$

$$5) v_{2,2} = \cos \frac{\pi}{2} = 0$$

$$6) v_{3,2} = \cos \frac{3\pi}{4} \approx -0,707$$

$$7) v_{4,2} = -1$$

$$8) v_{4,1} = -0,25$$

$$9-12) v_{4,0} = 0 = v_{3,0} = v_{2,0} = v_{1,0}$$

Начальное приближение $v^{(0)} = 0$

Итерация 1

1) узел $v_{1,1}$

По формуле (6):

$$v_{1,1}^{(1)} \approx \frac{\frac{0 + 0,25}{(\frac{\pi}{4})^2} + \frac{0 + 0,707}{(\frac{1}{2})^2} - (2 - \frac{1}{4}) \cos(\frac{\pi}{4})}{\frac{2}{(\frac{\pi}{4})^2} + \frac{2}{(\frac{1}{2})^2}} = 0,178$$

2) ужен $v_{2,1}$

$$v_{2,1}^{(1)} \approx \frac{\frac{0+0,178}{(\frac{\pi}{4})^2} + \frac{0+0}{(\frac{1}{2})^2}}{\frac{2}{(\frac{\pi}{4})^2} + \frac{2}{(\frac{1}{2})^2}} = 0,0257$$

3) ужен $v_{3,1}$

$$v_{3,1}^{(1)} \approx \frac{\frac{-0,25+0,0257}{(\frac{\pi}{4})^2} + \frac{0-0,707}{\frac{1}{4}} - (2-\frac{1}{4})\cos(\frac{3\pi}{4})}{11,242} = -0,174$$

$$v_{1,1}^{(1)} \approx 0,178; v_{2,1}^{(1)} \approx 0,0257; v_{3,1}^{(1)} \approx -0,174$$

Итерация 2

1) ужен $v_{1,1}$

$$v_{1,1}^{(2)} \approx \frac{\frac{0,0257+0,25}{0,617} + \frac{0+0,707}{\frac{1}{4}} - 1,75\cos(\frac{\pi}{4})}{11,242} = 0,1813$$

2) ужен $v_{2,1}$

$$v_{2,1}^{(2)} \approx \frac{\frac{-0,174+0,1813}{0,617}}{11,242} = 0,0011$$

3) ужен $v_{3,1}$

$$v_{3,1}^{(2)} \approx \frac{\frac{-0,25+0,0011}{0,617} + \frac{0-0,707}{\frac{1}{4}} - 1,75\cos(\frac{3\pi}{4})}{11,242} = -0,1774$$

$$v_{1,1}^{(2)} \approx 0,1813; v_{2,1}^{(2)} \approx 0,0011; v_{3,2}^{(2)} \approx -0,1774$$

5. Подготовка контрольных тестов для иллюстрации метода

1. Для исследования зависимости времени выполнения от размера сетки задан дискретный набор из двух групп по 20 расчетных сеток.:
 - Фиксированное значение $N_x = 40$ (и 100) при изменении N_y от 10 до 200 с целью изоляции влияния дискретизации по оси Y .
 - Фиксированное значение $N_y = 40$ (и 100) при изменении N_x от 10 до 200 с целью определения чувствительности алгоритма к декомпозиции полос по оси X .
2. Фиксированный нагрузочный тест масштабируемости: Сконфигурирован в функции `study_scaling_performance` на анизотропной сетке размерностью 80×400 узлов. Данный тест предназначен для циклического исполнения на варьируемом числе параллельных процессов MPI ($p \in [1, 12]$) с целью вычисления масштабируемости вычислительного ядра, ускорения S_p и эффективности E_p . Метрики сохраняются в файл `scaling_data.csv`.

6. Модульная структура программы

Дерево проекта приведено на [Листинге 1](#).

Перейдём к подробному описанию файлов и функций программного комплекса:

- `tools.c`
 - `double **alloc_matrix(int m, int n)` - выделение динамической памяти для матрицы размера $m \times n$;
 - `void free_matrix(double **A, int n)` - освобождение памяти, занятой матрицей из n строк;
 - `double max_in_matrix_diff(double** A, double** B, int m, int n)` - вычисление максимального модуля разности элементов двух матриц;
 - `void init_matrix(double** A, int m, int n, double value)` - инициализация всех элементов матрицы заданным значением;
 - `void swap_matrices(double **A, double **B)` - быстрый обмен указателей двух матриц текущей и предыдущей итераций.
- `seidel.c`
 - `double **seidel(double border_x, double border_y, int Nx, int Ny, double eps, ...)` - функция блочно-параллельного метода Зейделя с обменом фиктивными слоями через `MPI_Sendrecv` и глобальным сбором матрицы решения через `MPI_Gatherv`.


```

└─ src/
    ├── CMakeLists.txt
    ├── main.c
    ├── def.h
    ├── seidel.c
    ├── seidel.h
    ├── study.
    ├── study.h
    ├── test.
    ├── test.h
    ├── functions.
    ├── functions.h
    ├── tools.c
    ├── tools.h
    ├── build/
    │   └─ seidel
    └─ results/
        ├── scaling_data.csv
        ├── time_vs_grid.csv
        └─ solution.csv

```

Листинг 1: Структура программного комплекса

- `functions.c`

- `double f(double x, double y)` - вычисление значения функции правой части уравнения Пуассона;
- `double u_0y(double y), double u_piy(double y)` - задание аналитических граничных условий на левой и правой границах;
- `double u_x0(double x), double u_x1(double x)` - задание аналитических граничных условий на нижней и верхней границах.

- `study.c`

- `void study_time_vs_grid_size(int rank, int size)` - изучение зависимости времени выполнения метода от размеров расчетной сетки с записью результатов в `time_vs_grid.csv`;
- `void study_scaling_performance(int rank, int size)` - расчет времени, ускорения и эффективности параллелизации на фиксированной сетке 80×400 при изменении числа процессов от 1 до 12 с записью в `scaling_data.csv`.

- `test.c`

– `void run_test_case(int rank)` - однократный запуск алгоритма на сетке тестового примера **раздела 4**.

- `main.c`

– `int main(int argc, char **argv)` - инициализация параллельной среды MPI, запуск расчетных модулей и корректное освобождение ресурсов.

7. Численный анализ решения задачи

7.1. График решения

7.2. Исследование параллельной реализации

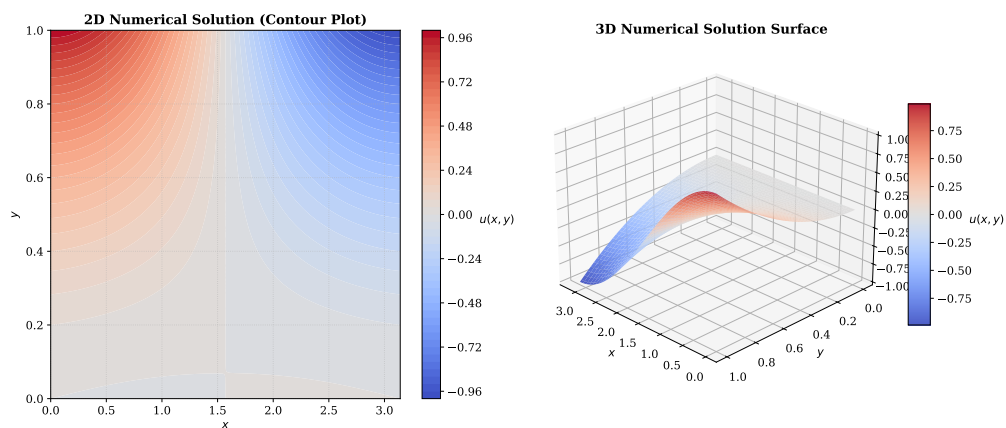


Рис. 7.1

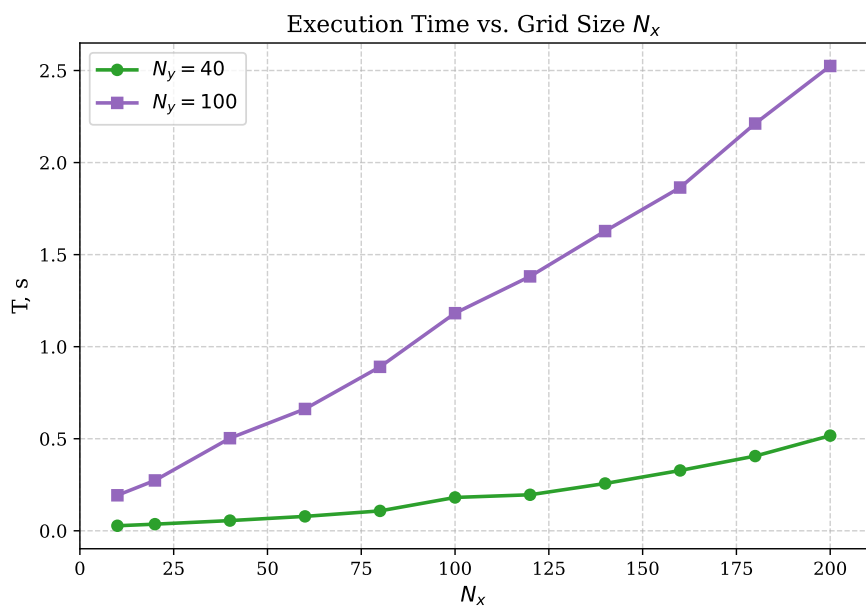


Рис. 7.2

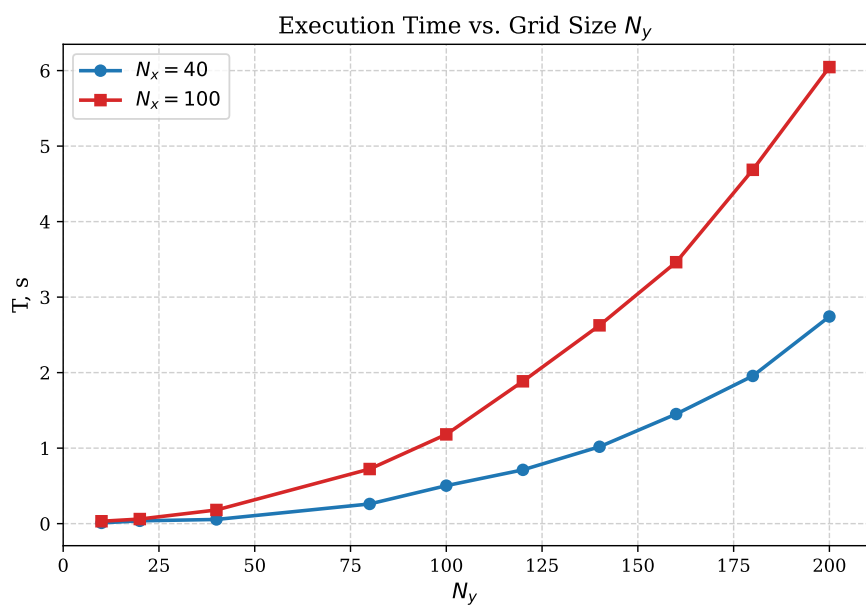


Рис. 7.3

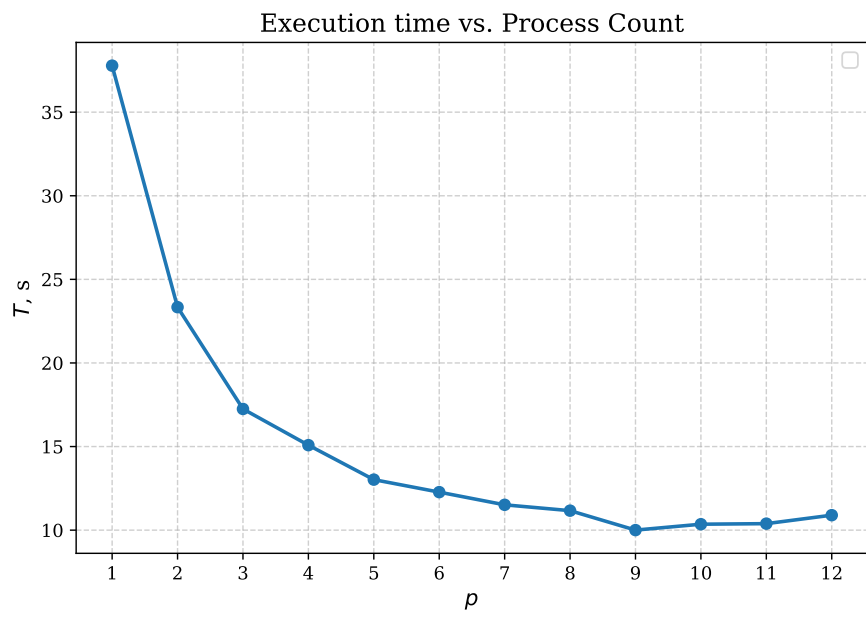


Рис. 7.4

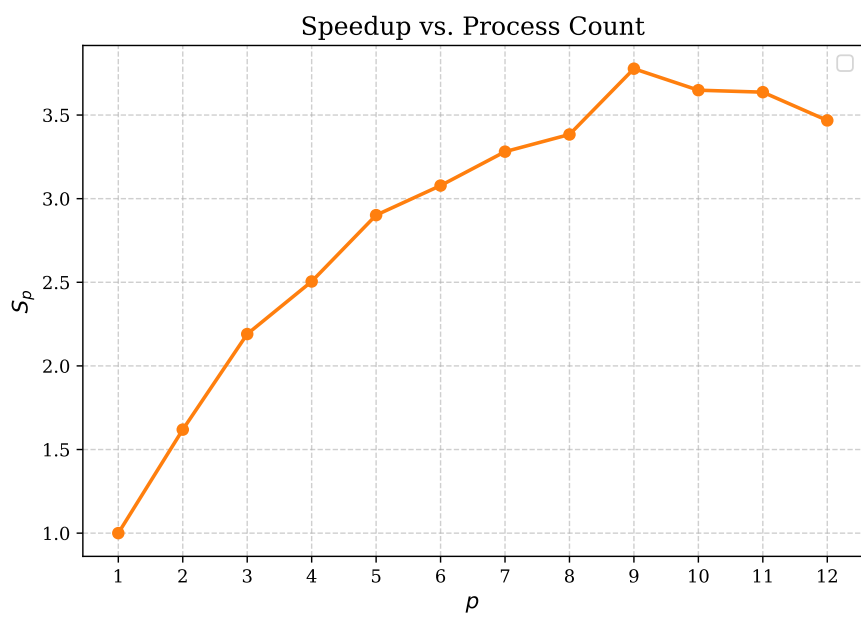


Рис. 7.5



Рис. 7.6

8. Выводы

9. Примечания